Lecture 7 : Stochastic Simulation

- Stochastic simulation is simulation involving random variation.
  - Contrast with deterministic simulation. You have all seen SIR (susceptible/infected/recovered) model simulations from deterministic ordinary differential equations since 2020!
- We examine why stochastic simulation is useful.
- We look at simulation methods in R.

What is the Use of Stochastic Simulation?

- Stochastic simulation can be used to learn about the properties of a statistical model when mathematical analysis proves intractable.
  - Suppose we cannot work out mean of a random variable, but can simulate it.
  - Then do so many times, and use sample mean as approximation to theoretical mean. But why is it OK?? See next slide.
- Modern computing power makes simulation feasible in many situations.
  - Though very complex models will require very efficient code; e.g. in genetics.
- Simulation can also be used as a convenient computational device when performing Bayesian statistical inference.

Why Stochastic Simulation is Reliable

- Stochastic simulation is reliable if we generate sufficiently many simulations.
- Theoretical support provided by the Law of Large Numbers.

Theorem (Law of Large Numbers (Weak Version))

Let $X_1, \ldots, X_n$ be a random sample (i.e. a sample of independent and identically distributed random variables) from a distribution with mean μ and finite standard deviation. Define $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$ to be the sample mean. Then $\bar{X} \xrightarrow{p} \mu$ as $n \to \infty$, where $\xrightarrow{p}$ indicates convergence in probability.

- No need to worry too much about the technical details of convergence in probability.

Do Computers Roll Dice?

Table of random numbers 1 to 100 with no numbers repeated in each block of 25.

75 64 26 45 10	79 18 58 61 09	67 05 60 19 91	14 62 02 35 98	88 51 53 56 96
24 05 89 42 27	98 62 31 19 95	24 25 58 50 49	19 30 31 58 59	49 47 85 48 30
63 18 80 72 41	26 11 91 96 81	55 92 44 23 93	97 89 53 40 80	29 46 34 39 63
38 81 93 68 22	84 92 59 82 80	26 94 73 71 45	63 84 68 44 94	93 64 13 94 31
25 59 54 43 02	16 41 97 40 65	70 29 77 74 27	69 81 70 01 95	82 99 77 80 21
12 28 15 88 98	21 28 92 06 08	33 72 05 13 06	85 65 33 90 20	92 33 27 59 49
36 59 95 67 96	25 72 30 41 81	71 92 18 65 17	64 58 56 89 28	69 18 36 06 71
91 72 33 68 11	22 20 15 01 65	34 60 47 16 09	44 45 46 97 83	44 51 98 67 29
86 04 47 43 69	12 85 04 93 74	80 08 57 25 79	72 96 07 57 40	82 62 68 60 73
01 05 65 97 77	96 64 98 62 49	07 19 63 46 66	77 98 80 54 60	97 32 83 74 80
26 95 96 93 87	17 59 90 35 94	73 68 03 27 29	49 64 66 14 65	57 24 45 76 39
45 27 71 62 05	71 18 32 42 91	25 66 46 49 71	67 11 25 23 12	41 47 99 66 01
74 07 90 20 25	05 52 65 84 92	87 57 95 37 83	85 45 22 56 26	10 28 04 88 49
77 99 91 43 02	96 06 07 36 68	17 48 06 09 84	31 86 91 87 96	63 87 32 33 70
75 53 35 46 41	21 95 85 61 46	94 18 78 39 47	19 60 48 15 59	68 79 42 09 67
45 65 84 36 28	48 33 82 62 71	74 48 75 92 34	32 94 26 70 88	35 50 19 97 52
81 74 60 90 46	13 51 24 54 55	45 54 12 90 99	44 68 86 71 58	27 51 81 11 77
95 11 96 85 83	93 53 74 52 97	79 53 21 41 44	45 81 02 38 07	38 07 80 89 56
29 40 82 33 86	67 95 43 41 89	05 52 17 31 13	82 61 78 57 40	84 39 57 63 78
79 14 32 21 09	32 27 02 70 20	61 47 24 42 76	77 27 99 36 15	36 98 08 40 53
51 46 23 17 11	93 35 70 37 86	26 23 64 88 17	17 78 95 93 83	65 23 90 78 55
98 75 60 99 89	91 18 20 27 74	31 82 01 32 97	97 43 21 87 62	33 28 10 56 98
15 97 42 56 79	08 58 79 40 31	37 19 20 58 41	41 86 66 54 45	08 76 89 86 32
06 16 35 93 26	36 97 26 17 71	74 95 89 06 50	50 62 48 46 26	24 95 93 01 64
54 43 55 21 74	47 59 75 03 57	63 38 02 51 77	77 76 65 08 92	72 29 35 06 85

MASSEY UNIVERSITY
TE KUNENGA KI PŪREHUROA
UNIVERSITY OF NEW ZEALAND

Do Computers Roll Dice?

- Hiccup: computers are entirely deterministic machines.
- For a given set of inputs, they will always return precisely the same output.
- So how can computers produce random numbers?
- The answer is: they cannot!?!
- Instead they produce **pseudo-random numbers**.
- These are sequences that look random to the naked eye, and indeed will pass any standard statistical test for randomness.
- Nonetheless, they are generated by complicated but entirely deterministic algorithms; e.g. **congruential random number generators**.

(Pseudo) Random Number Generation in R

- R is equipped with some of the best available (pseudo-)random number generators.
- Underlying generators produce uniform random variables which are transformed to produce simulations from any of the common probability distributions.
- R function name formed by adding the prefix `r` to the requisite R distribution name.
  - `rpois` generates simulations from the Poisson distribution,
  - `rexp` produces values from the exponential distribution.

Setting the Seed for Reproducibility

- Because pseudo-random number generators use deterministic algorithms, it is possible to reproduce the results.
- Can be achieved using the `set.seed` command.
  - The `set.seed` function takes a single integer as its argument.
- This initializes the random number generator in a particular state.
- Re-running any given simulation code with the same seed will result in same values being generated.
- Important for **reproducible research**, since it means statistical analyses involving simulation can be verified.

Setting the Seed

R Snippet

```
> rexp(5,rate=2)
[1] 0.427737086 0.617941673 0.871556954 0.006092774 0.29644
> rexp(5,rate=2)
[1] 1.29511829 1.00355363 1.57605795 0.00417631 0.18680190
> set.seed(13579)
> rexp(5,rate=2)
[1] 1.07615712 0.11218861 1.04711636 0.03843151 0.20684597
> set.seed(13579)
> rexp(5,rate=2)
[1] 1.07615712 0.11218861 1.04711636 0.03843151 0.20684597
```

Example continued

Points to note

- Function `rexp(5,rate=2)` generates 5 random variables from the $\mathrm{Exp}(\lambda)$ distribution with $\lambda = 2$ (the 'rate parameter').
- Notice that first two sets of simulated values are different.
- Third and fourth set are identical, because they are generated using the same seed.
- Nothing special about the value 13479 for the seed.
- You can choose whatever integer you like (up to some huge maximum value).
  - E.g. `set.seed(10)` and `set.seed(333333)` both OK.

Example: simulation to evaluate model properties

- Suppose that we did not know how to compute the mean of the $\mathrm{Exp}(\lambda)$ distribution with $\lambda = 2$.
  - In fact we know that the mean is $1/\lambda = 0.5$.
- In this Example we derive an approximation by simulating *n* values from this distribution, and then calculating the sample mean for these.

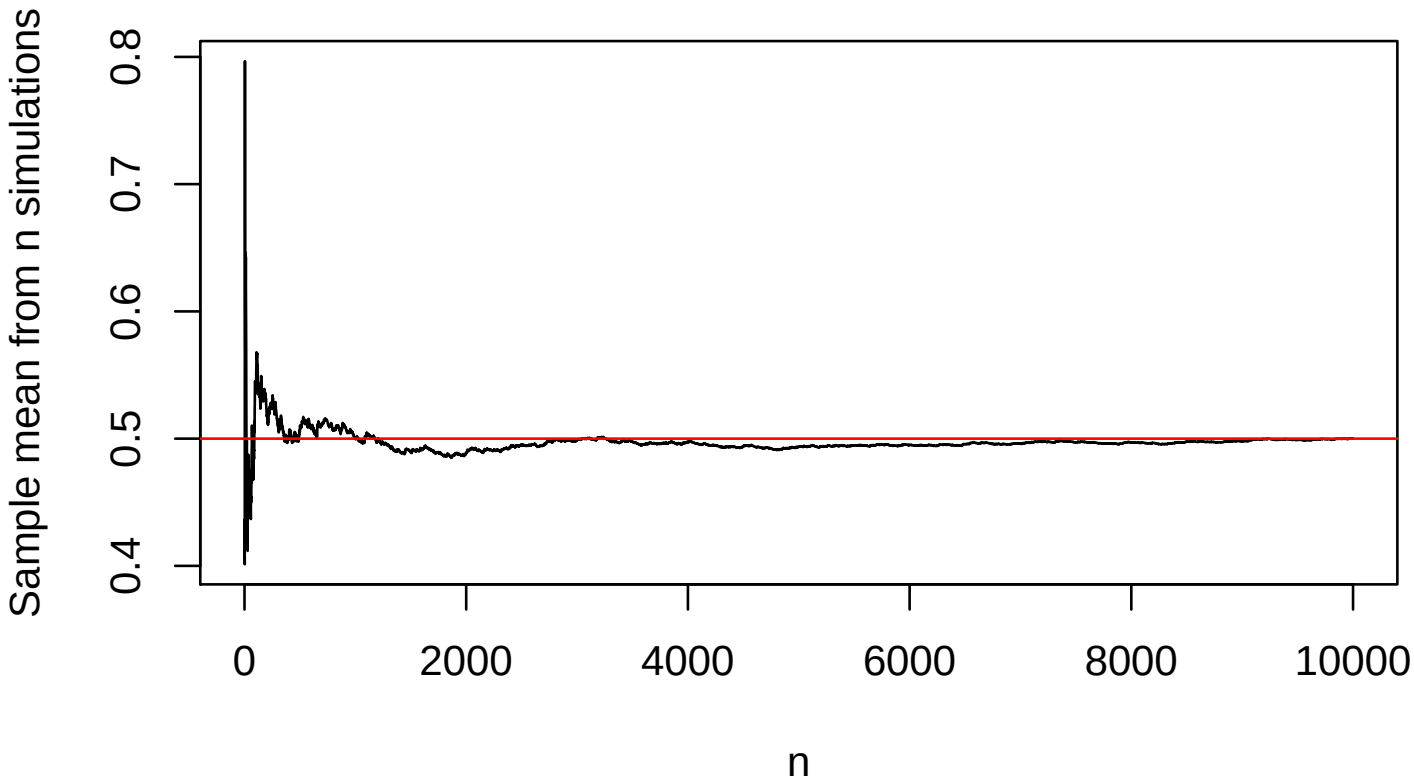
Example continued

R Snippet

```
> set.seed(29)
> mean(rexp(100,rate=2))
[1] 0.5353284
> mean(rexp(1000000,rate=2))
[1] 0.499813
> n <- 10000
> x <- rexp(n,rate=2)
> x.cumsum <- cumsum(x)
> x.means <- x.cumsum/(1:n)
> plot(x.means,type="l",xlab="n",ylab="Sample mean from n s
> abline(h=0.5,col=2)
```

Example continued

Figure



- Cumulative moving average from *n* simulations.
- The red horizontal line represents true distribution mean.

MASSEY UNIVERSITY
TE KUNENGA KI PŪREHUROA
UNIVERSITY OF NEW ZEALAND

Example continued

Points to note

- Set the seed so that results are reproducible.
- Second and third commands obtain estimates of the distribution mean based on 100 and $1,000,000$ simulations respectively.
  - Second value particularly closes to the true mean of $1/\lambda = 0.5$.
- `cumsum(x)` calculates the **cumulative sum** of `x`.
  - That is, the mth element of `cumsum(x)` is the sum of the first m elements of `x`.
- The mth element of `1:n` is `m`. If follows that the mth element of `x.cumsum/(1:n)` is given by $(\sum_{i=1}^{m} x_i)/m$, the sample mean based on the first m values.
- Hence `x.means` contains the cumulative moving average for the n simulations.

Example continued

Further points to note

- Cumulative moving average is plotted in Figure.
- Horizontal red line shows theoretical mean.
  - Added using the syntax `abline(h=0.5,col=2)`.
  - `h=0.5` indicates that a line should be drawn at height 0.5.
  - `col=2` defines the plotting colour to be red.
- Graph illustrates Law of Large Numbers in practice.
  - When n is small, sample mean can be some way distant from the distribution mean.
  - Sample mean convergences to the value 0.5 as n increases.

Counts rather than probability computations

- Example of the vampire identification test at the start of Chapter 3 of SR.
- More intuitive to be reasoning with a *natural frequency format*.
- We could draw our inspiration from this and sample from priors, posteriors, etc. to *see* predictions and understand the underlying distributions by *counting*.
- Notice that, in this case, we work on model uncertainty, not that of the real-world.
- Kill two birds with one stone: many scientists are uncomfortable with integral calculus in spite of a strong and valid intuition on how to summarize data and (ii) one of the most advanced method for posterior approximation (MCMC) produces nothing more than samples, so better know how to make sense of those now.

- Read the "Why statistics can't save bad science" box on p51 of SR.